

Virtual Reality (VR) Meeting Room



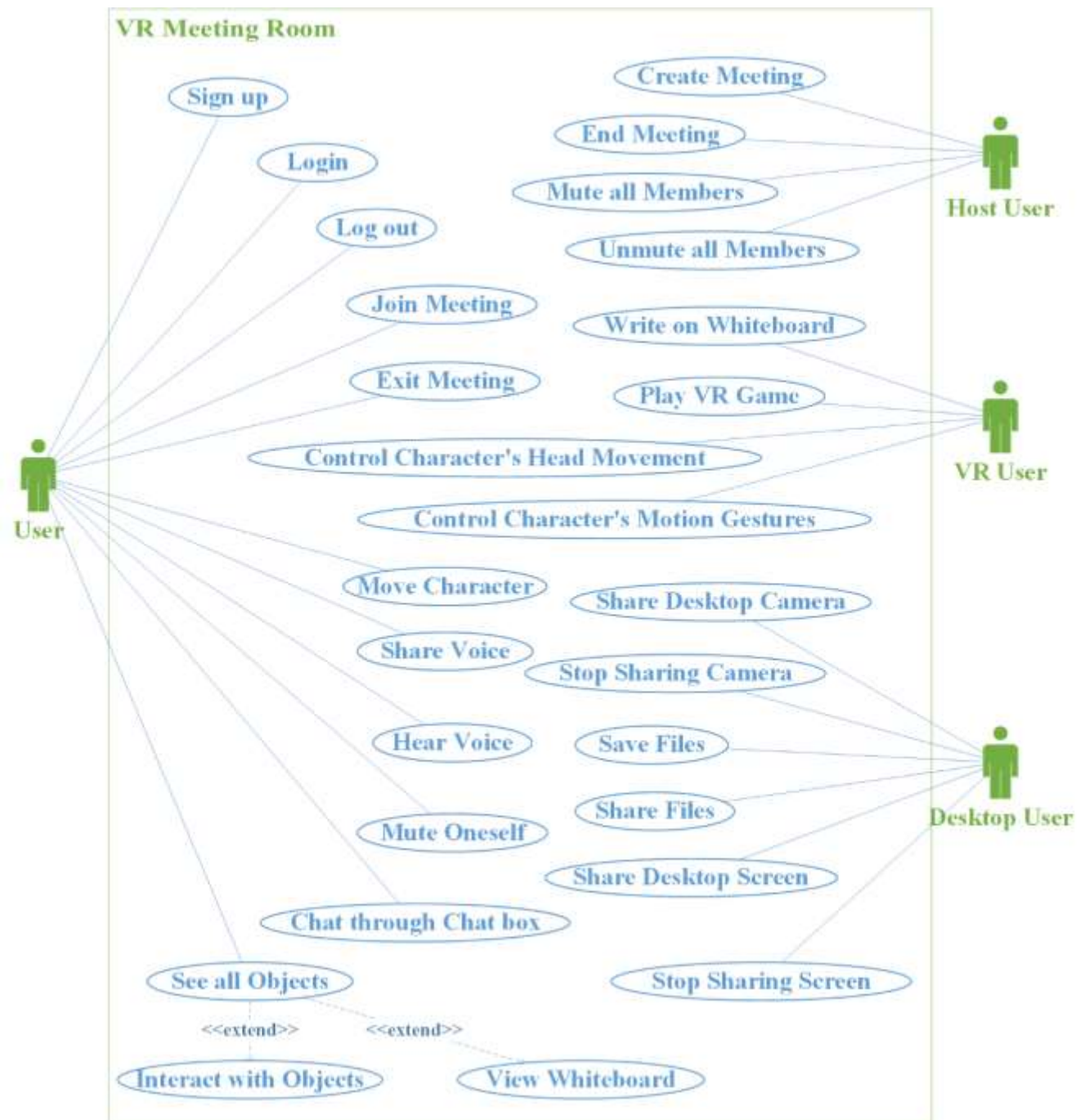
Introduction

User

Host User

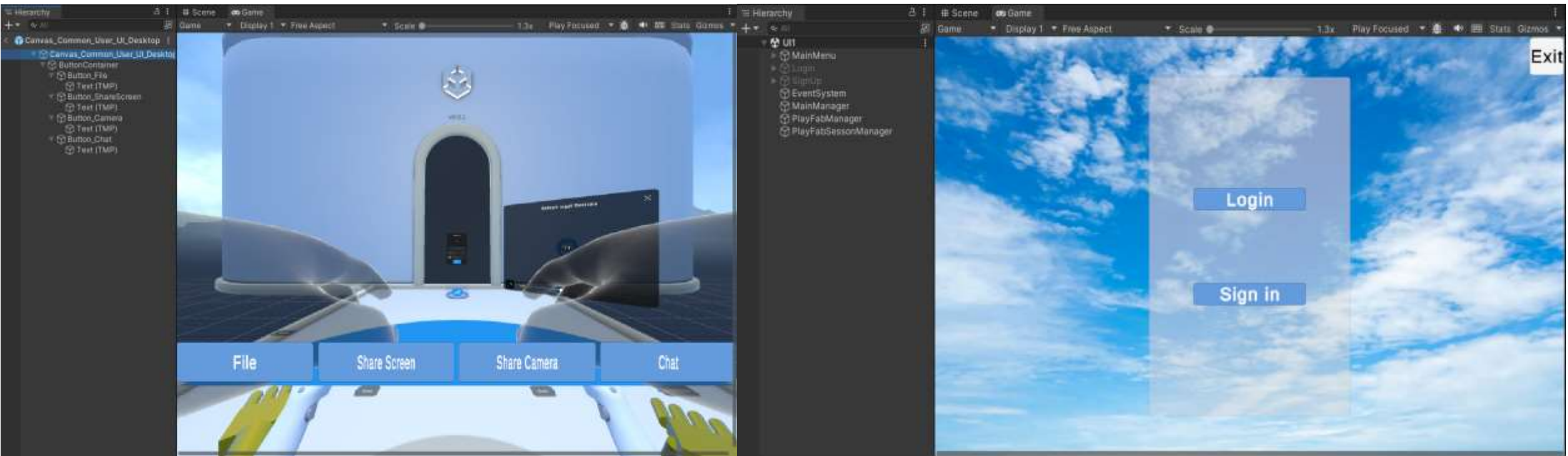
VR User

Desktop User



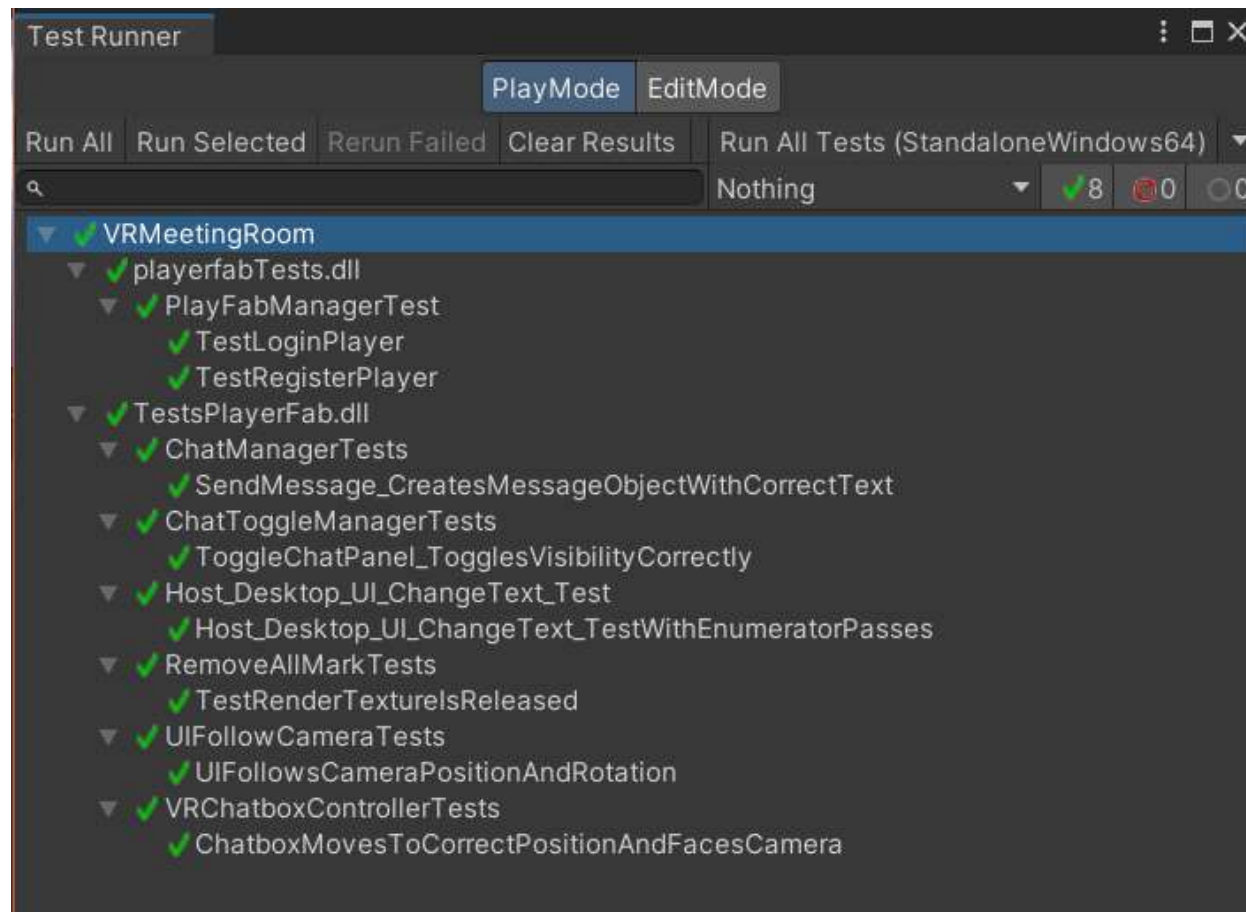
UI/UX Construction

- We used Unity's uGUI system in combination with TextMesh Pro to build the UI.



Unity Test Framework

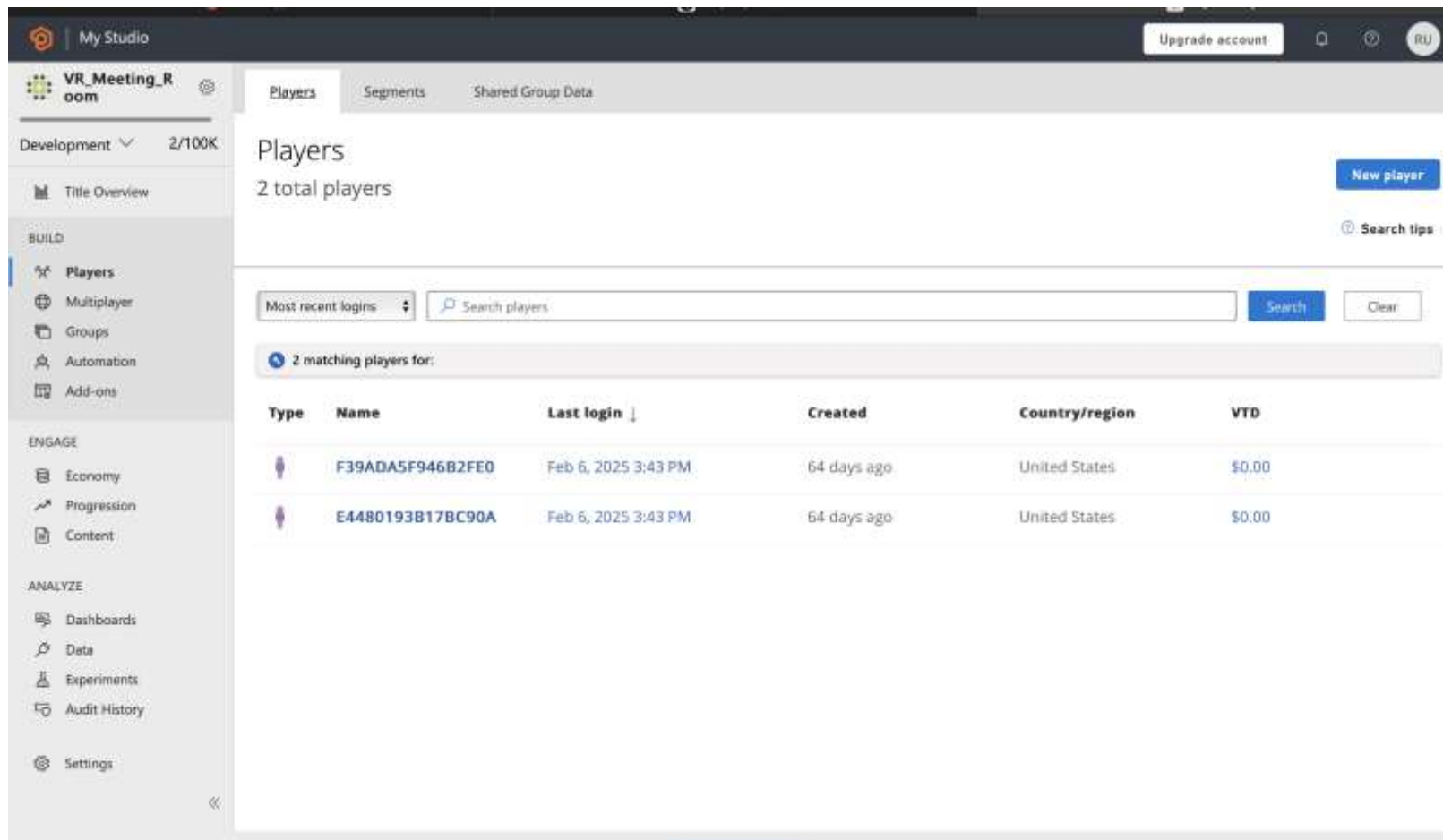
- For Test-Driven Development, we used the Unity Test Framework (UTF) to write test cases before writing any C# scripts.



Login、signup & database construction

Core Concept:

Integrate Unity with PlayFab server, which includes a built-in database, to support user registration, authentication, and file management functionalities.



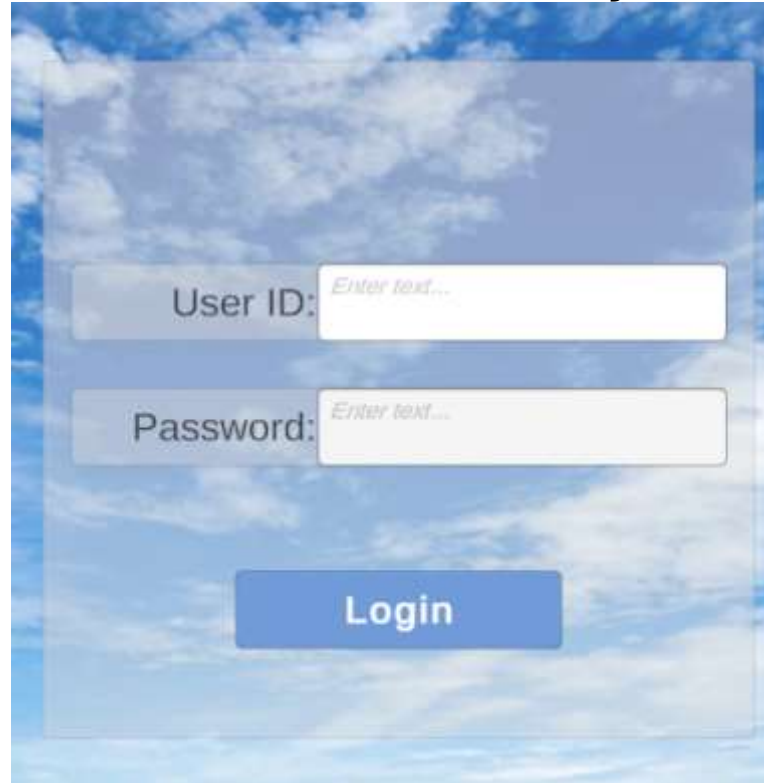
Login、signup & database construction

Core Implementation:

Unity communicates with the PlayFab database through centralized management by *PlayFabManager*, leveraging *PlayFabClientAPI.RegisterPlayFabUser* and *PlayFabClientAPI.LoginWithPlayFab* APIs. Usernames and passwords are passed from Unity as request data, and after PlayFab processes the request, a callback is sent back to Unity.



A sign-up form with a light blue sky background. It contains three input fields: 'User ID:', 'Password:', and 'Repeat Password:'. Each field has a placeholder text 'Enter text...'. Below the fields is a blue button labeled 'Sign Up'.

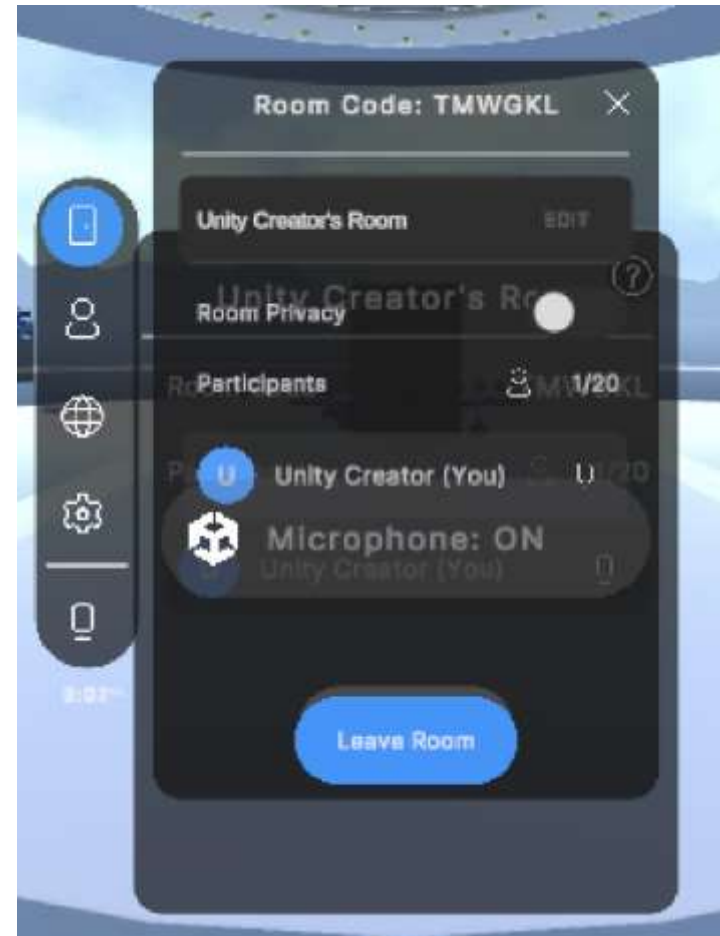
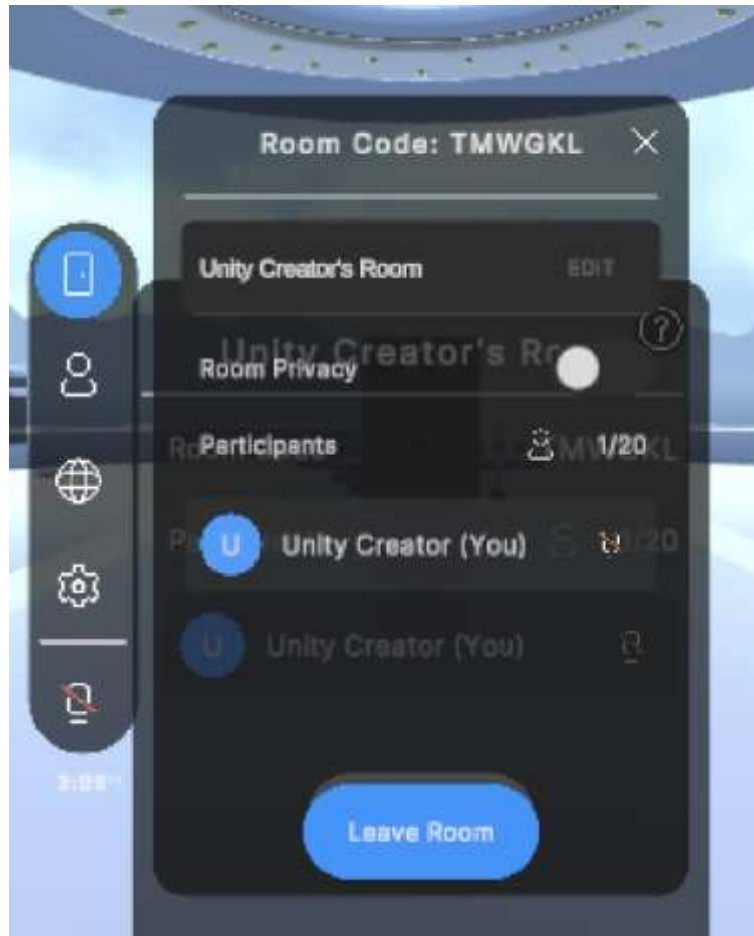


A login form with a light blue sky background. It contains two input fields: 'User ID:' and 'Password:'. Each field has a placeholder text 'Enter text...'. Below the fields is a blue button labeled 'Login'.

Mute&Unmute

1.Mute&Unmute oneself

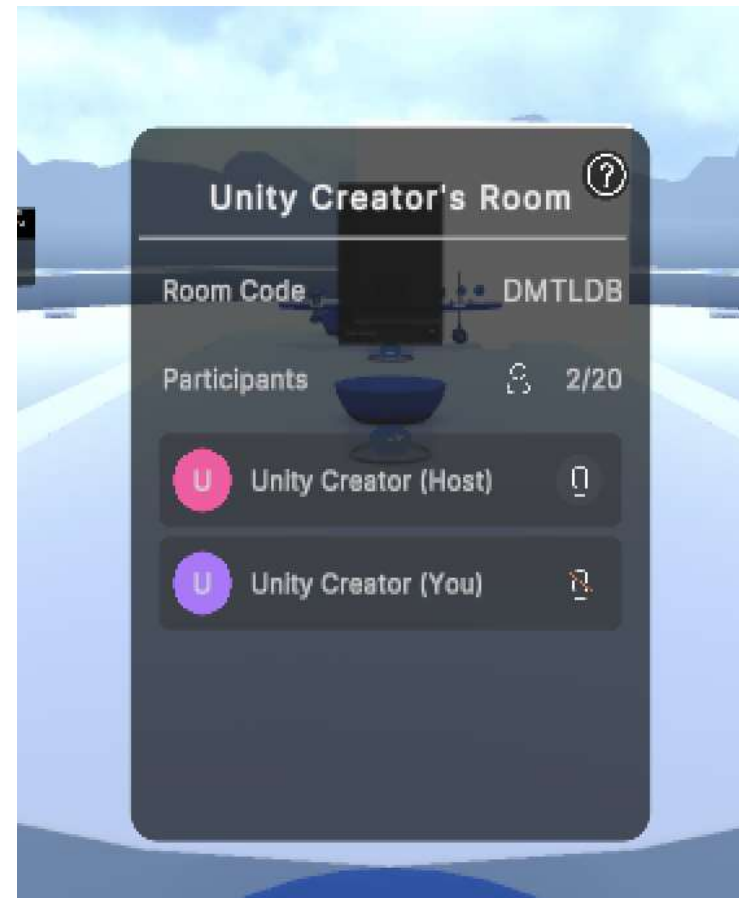
When the user presses the "M" key, the menu will pop up. By clicking the microphone icon, the user can mute or unmute.



Mute&Unmute

2.Mute&Unmute all members

This feature is only available to the host, so other participants will not see the button after joining the meeting.



Mute&Unmute

3.Core Implementation

Using the voice interface provided by Unity VR, permission control is implemented through the UI to enable mute and unmute functions.

```
public class MuteManager : MonoBehaviour
{
    private void Update()
    {
        if (NetworkManager.Singleton.IsHost)
        {
            canvas.enabled = true;
        }
        else
        {
            canvas.enabled = false;
        }
    }

    private void OnMuteAllChanged()
    {
        foreach (var player in FindObjectsOfType<XRINetworkPlayer>())
        {
            player.RequestMuteChangeServerRpc(!muted);
        }

        XRINetworkPlayer hostPlayer = FindObjectsOfType<XRINetworkPlayer>().FirstOrDefault(p : XRINetw
        if (hostPlayer != null)
        {
            hostPlayer.UpdateMuteStatusClientRpc(!muted);
        }

        playerListUI.MuteAllPlayer(!muted);
        muted = !muted;
    }
}
```

Share and Save Files

User clicks the File button to bring up the file upload and save interface

Click the Upload File button to select a file and upload it

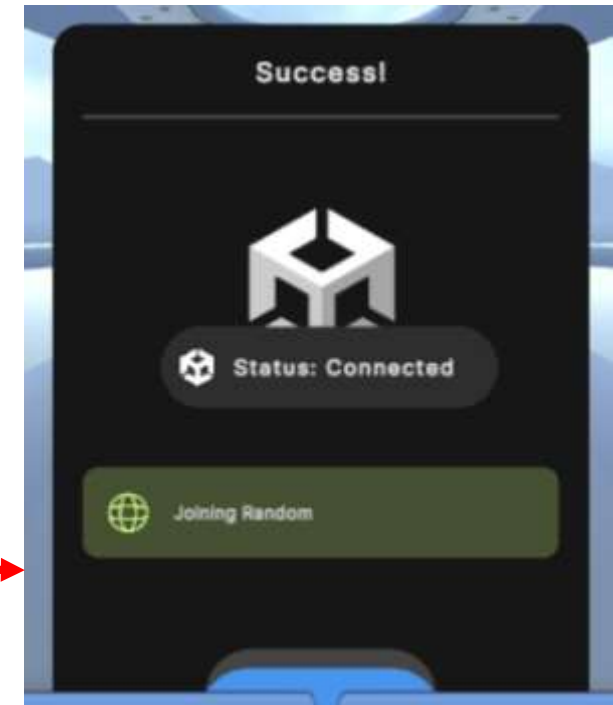
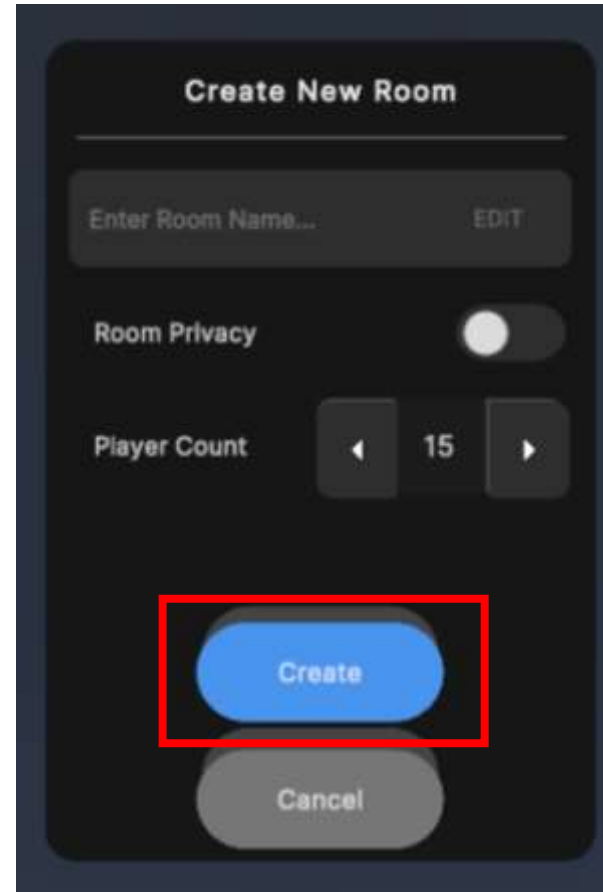
Click the (re) Load Files button to save or delete files

All functions implemented by the API provided by PlayFab to Unity.



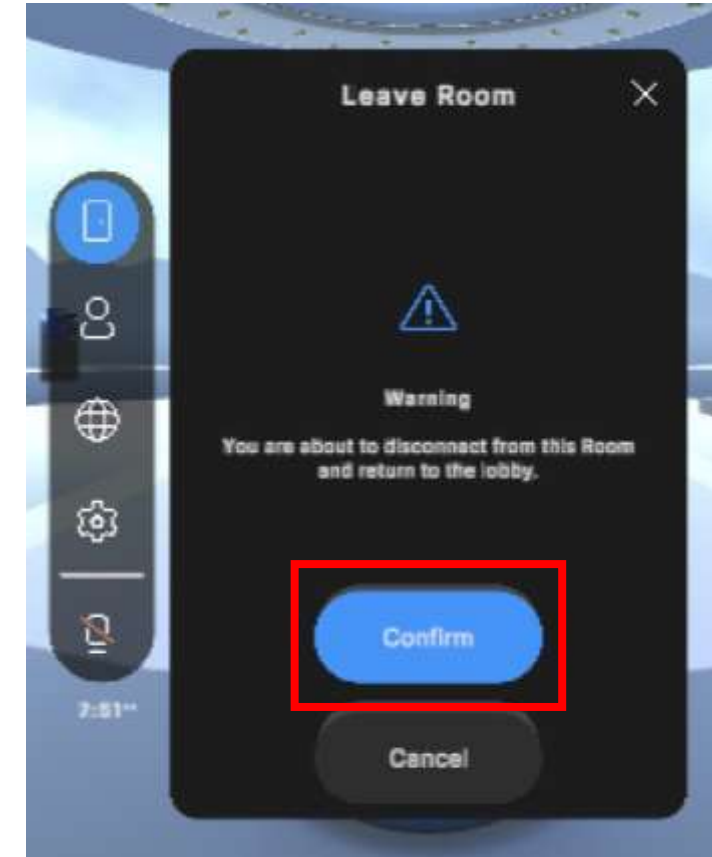
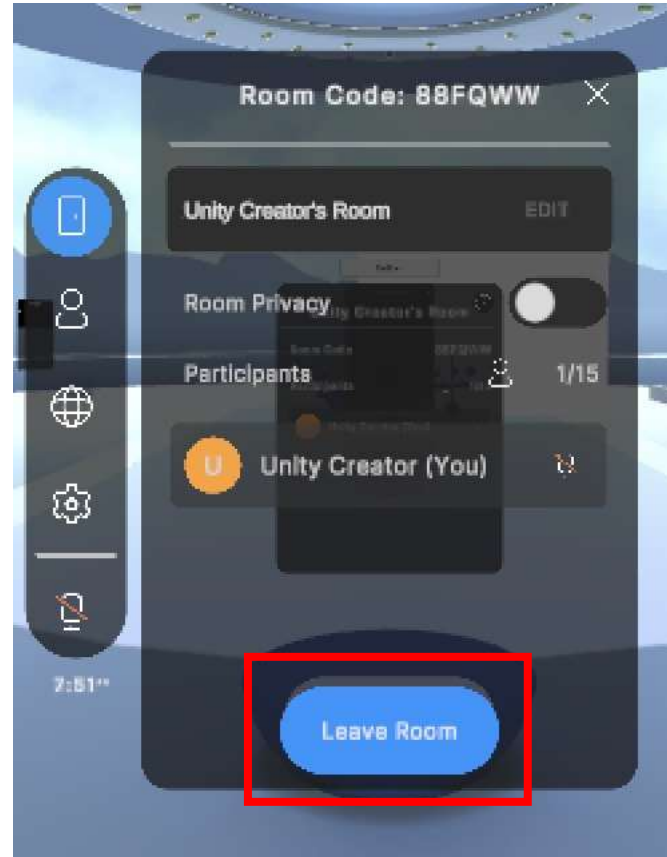
Meeting operations(Create/End)

- After the user selects “Create Meeting”, a meeting creation request is initiated via *PlayFabCreateMeetingRequest*. The initialization script is executed automatically, automatically loading the meeting room scene prefab, jumping into the scene and initializing the user's default position.
- Only the host user can end the meeting. After ending, the host user will jump to the lobby scene, while other users will jump to their own meeting room scene.

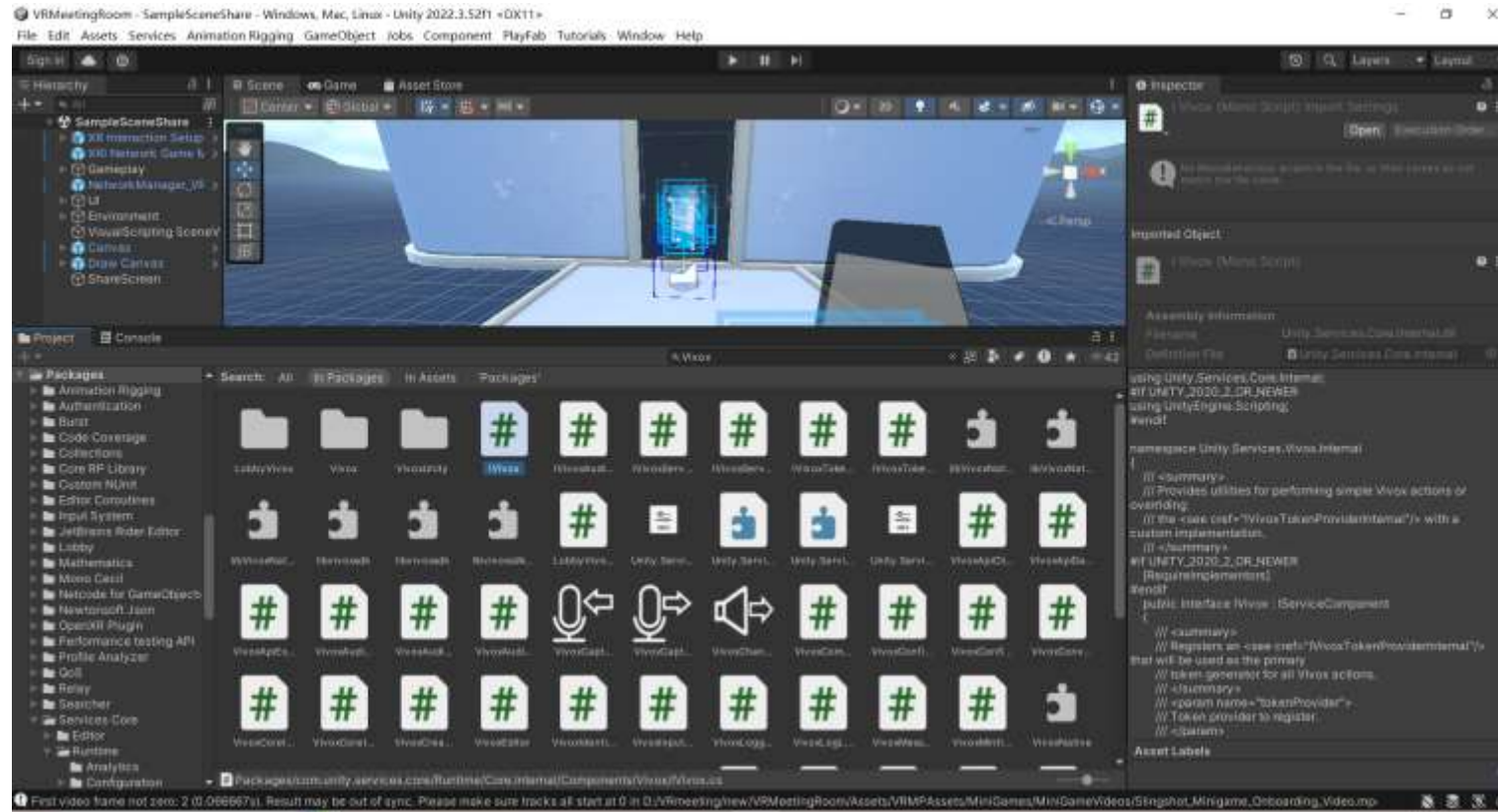


Meeting operations(Join/Exit)

- When starting a room, the script generates a random string as a unique room ID, which is the meeting number. We visualize the meeting number and display it in the UI in real time. Others can join the meeting by entering the meeting number.
- When the user selects “Exit Meeting”, the screen will automatically jump to the lobby scene.

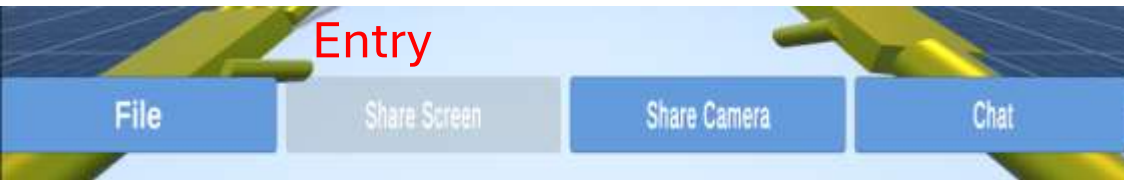


Share/Hear Voice

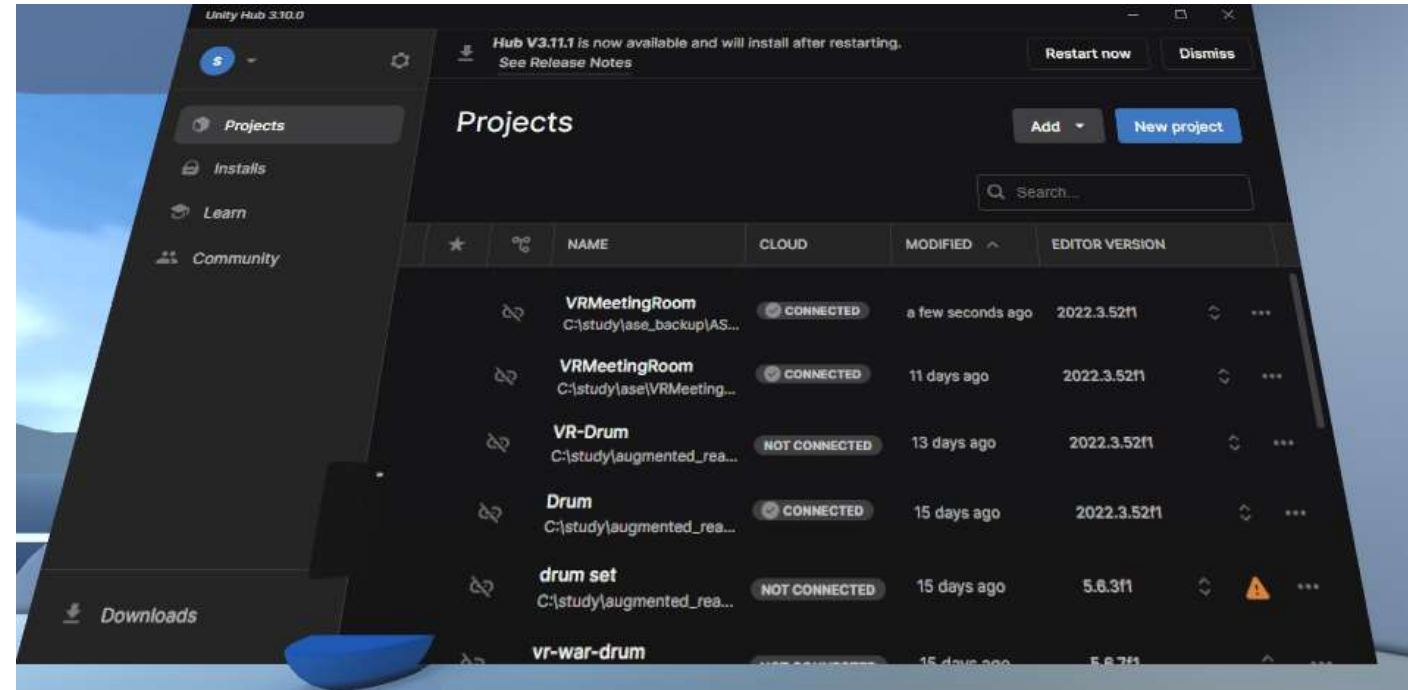


- We use a service platform called Unity Cloud, which integrates Vivox, a real-time voice SDK with very high quality, low latency, and support for additional features such as on-mic, mute, and volume adjustment. We have based various implementations of sound on this.

Share Screen

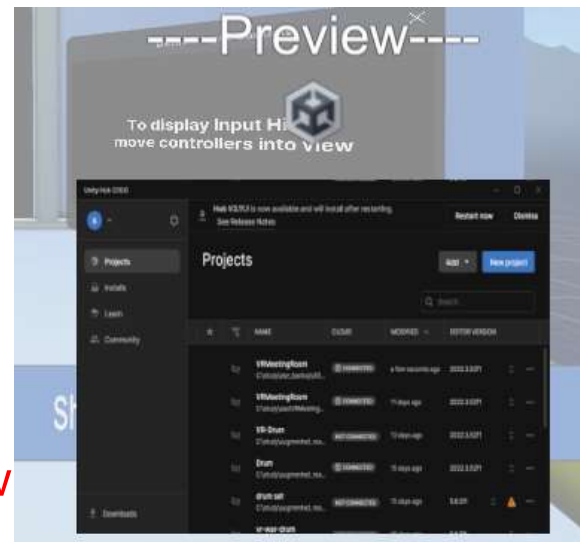


Select
window



Confirm and share
screen

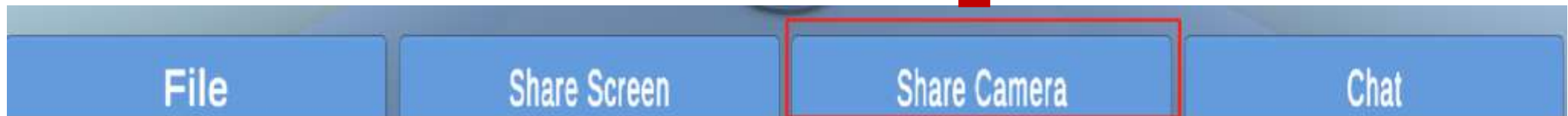
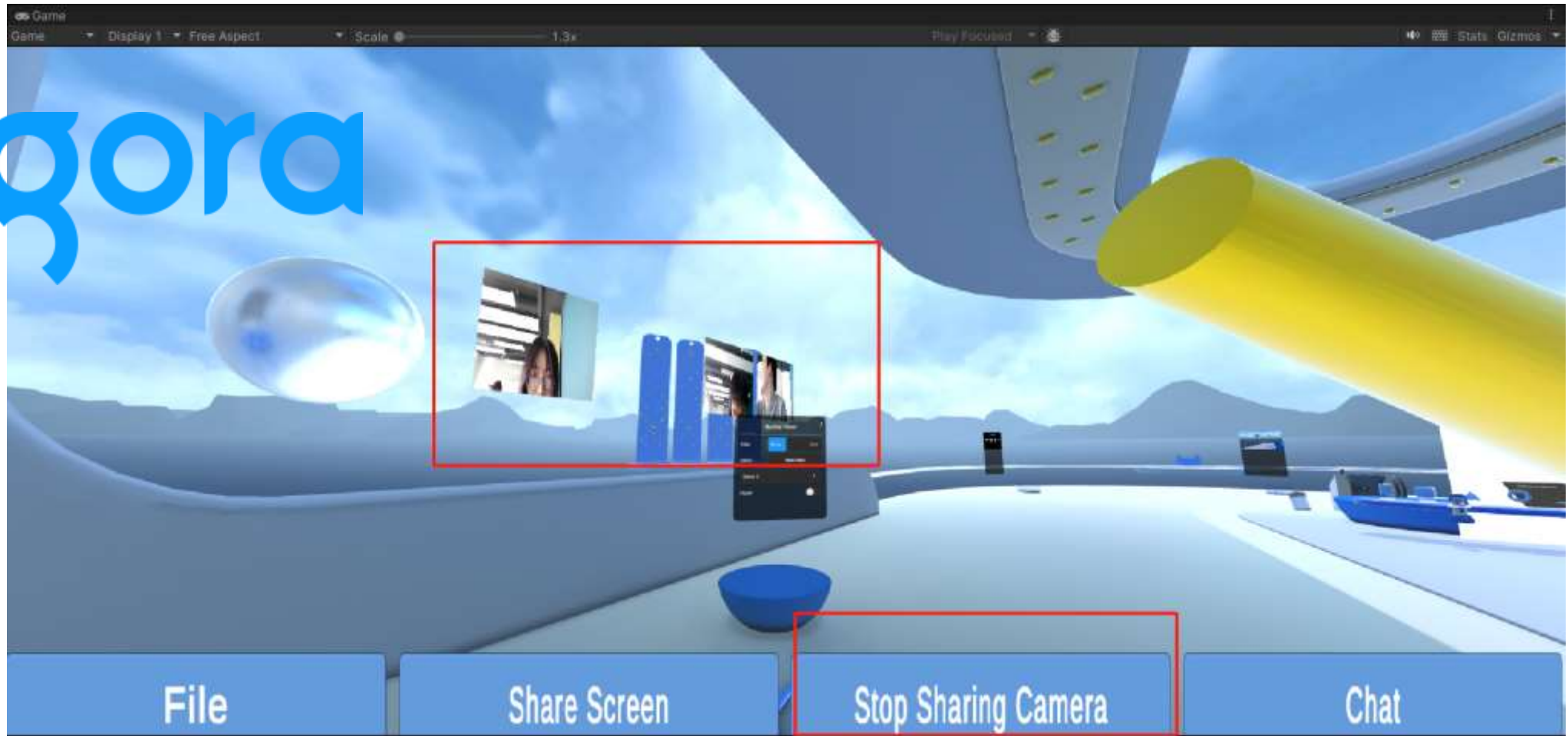
preview



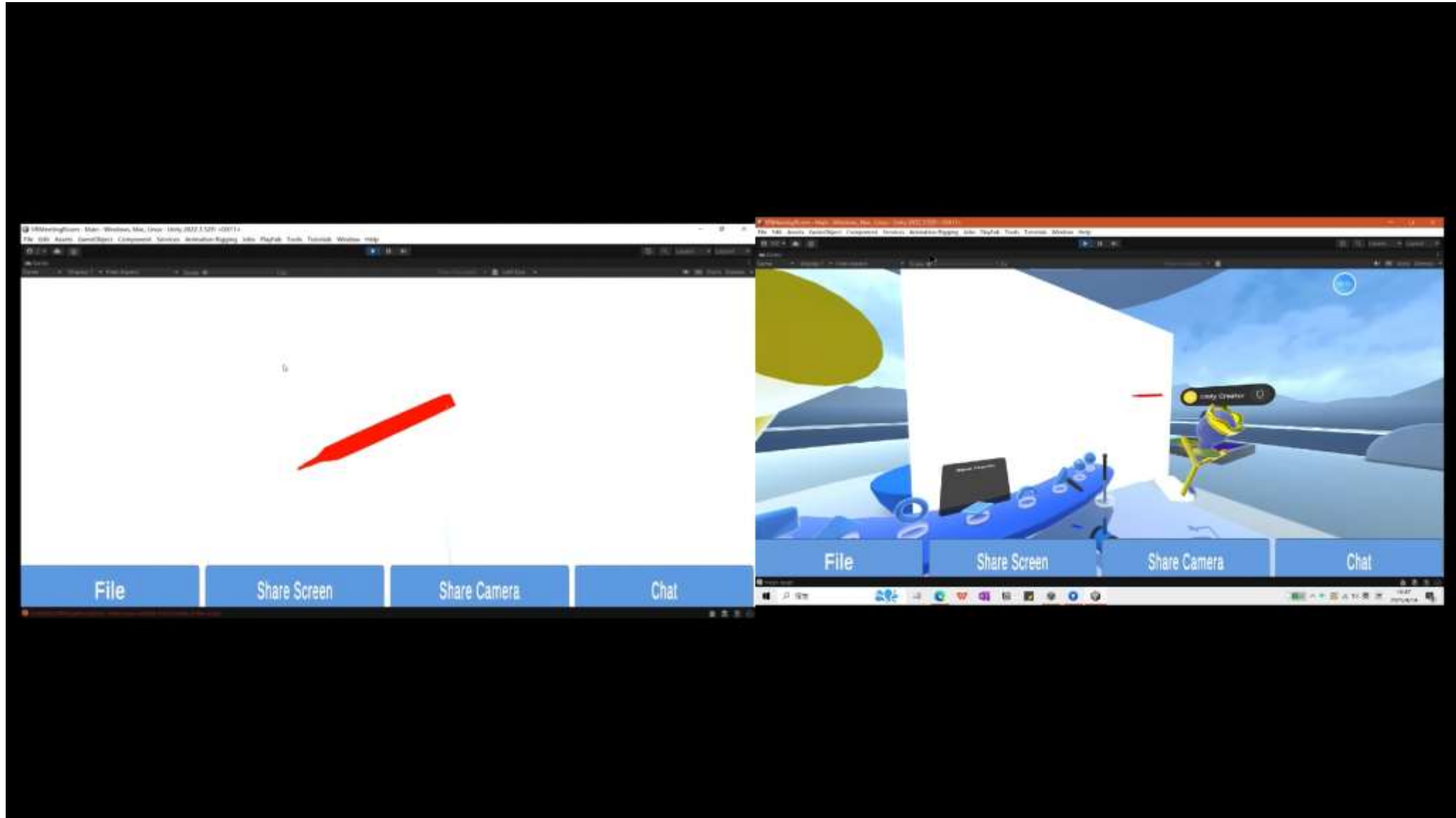
agora

Share Camera

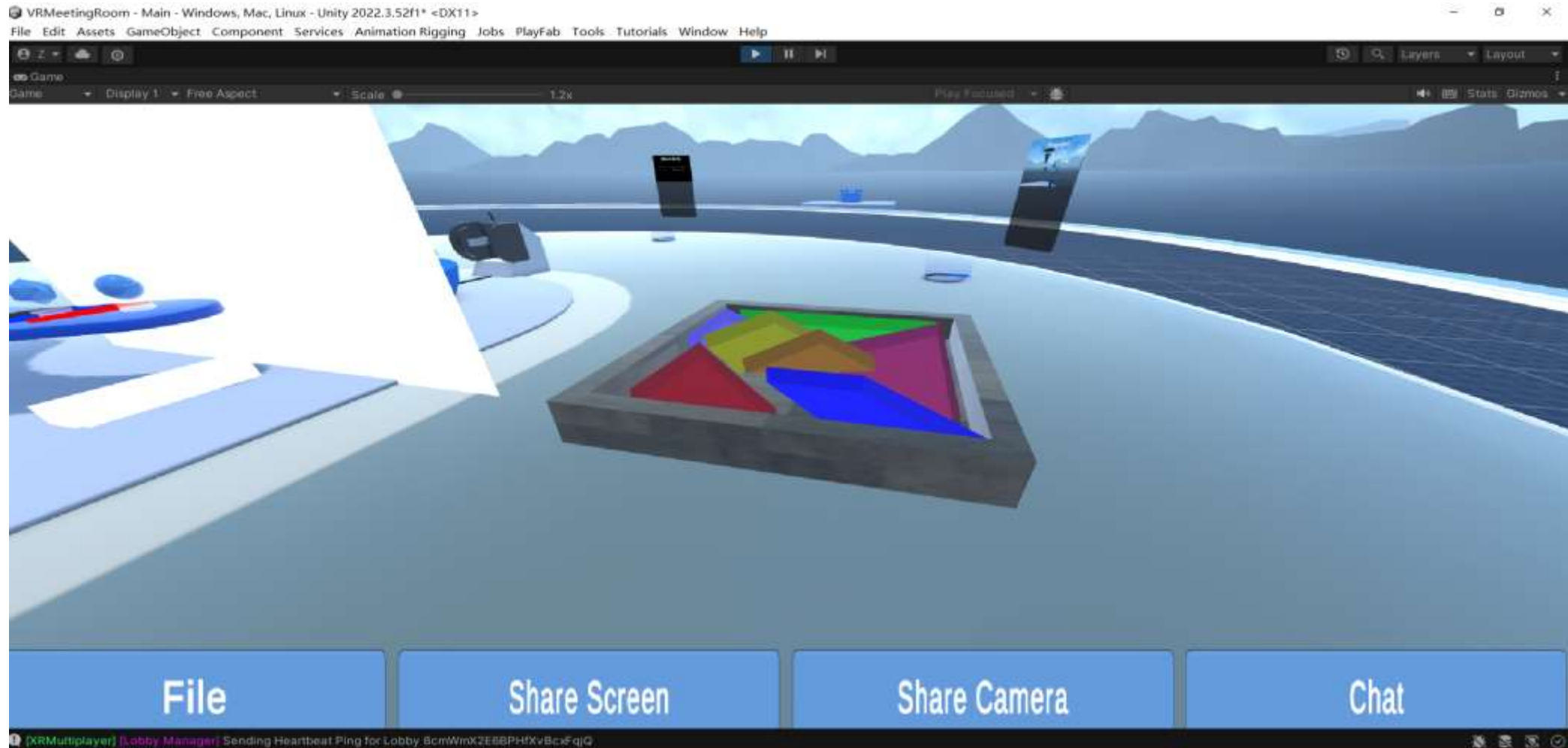
agora



DEMO

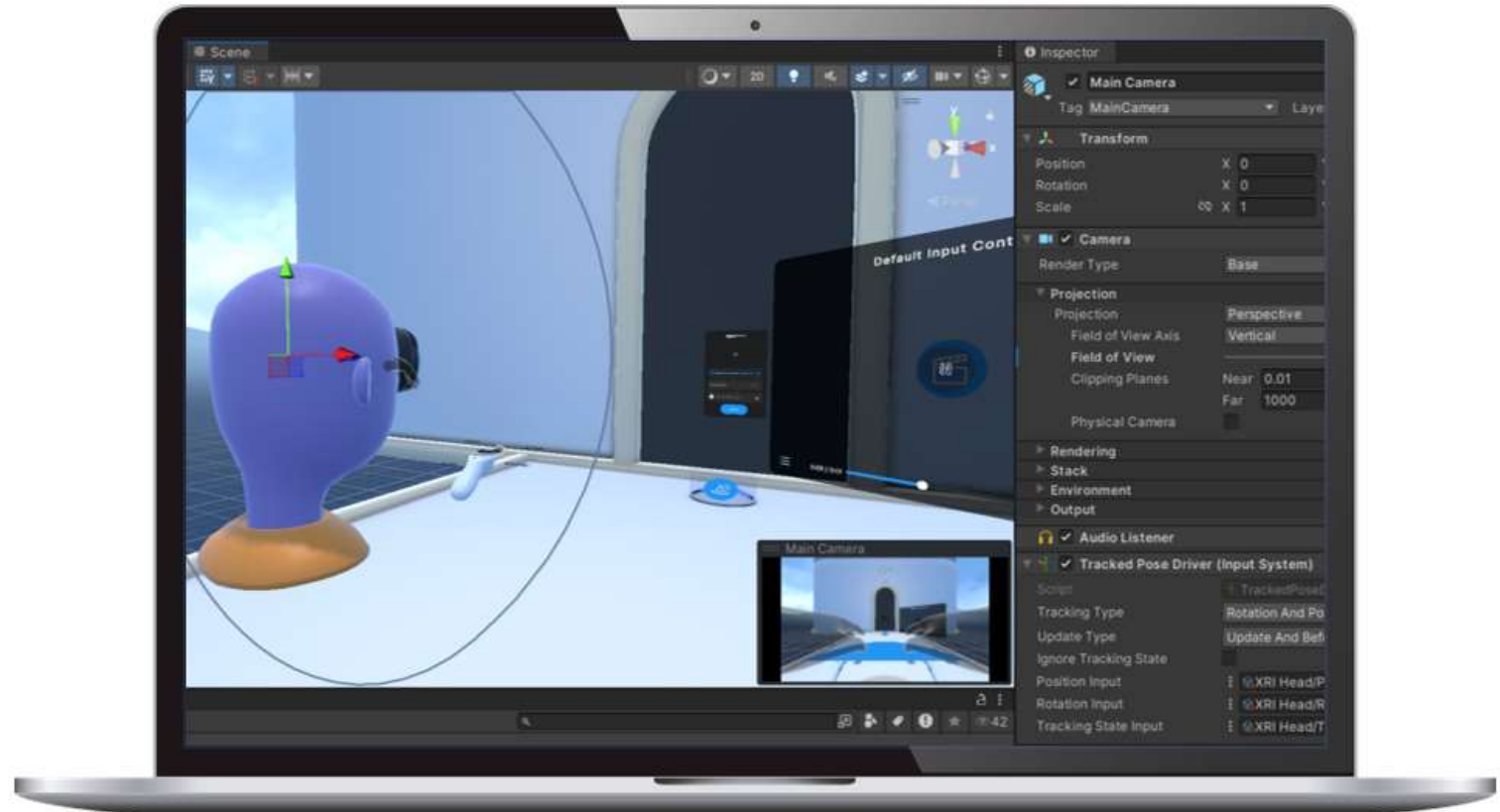


VR Game Seven Tangrams

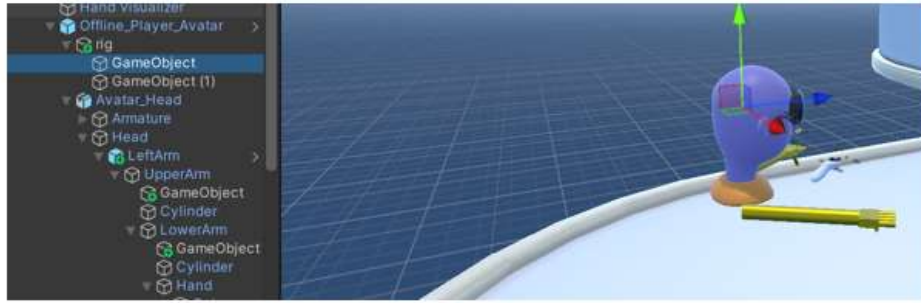


Control Head Movement

The project's head control was implemented by the Unity XR plugin, which allows us to turn our head to control the view by connecting a VR headset.



Control Motion Gestures



This feature is implemented using UnityXR and Animation Rigging components. With UnityXR, we can locate the position of the VR controller grips. Using Animation Rigging to create an IK system, the previously acquired VR controller grips position is used as the target input, and the IK system adjusts the bone chain (hand->LowerArm->UpperArm) from back to front according to the target's position, so that the movements of the characters in the virtual scene are synchronized with reality.

